COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate route for games

04

Lab and problem workbook

Practice, integration and assessment

20 lab contracts · 40 additional exercises · 25 module reviews · Five integrating projects

PACKAGE 04 / EXPANDED VOLUME 03 / SEPTEMBER 28, 2026

Independent study, implementation, evidence and review.

Contents

Lab and problem workbook 3

Module 11 — Native programming with C++ 4

Module 12 — Ownership, abstractions and C++ design 10

Module 13 — Data structures and their costs 15

Module 14 — Algorithms for maps, simulation and tools 20

Module 15 — Programming languages, compilers and computability 25

Sources and provenance 30

Lab and problem workbook

Use the complete contract

The 20 starter files provide first interfaces and a few checks, not complete projects. A starter compiling does not mean its lab passes. Complete the original requirement, the expansion steps, the listed cases, the independent transfer and the primary reading. Add missing tests yourself, then compare to the complete reference example after an independent attempt.

This workbook also supplies eight extra exercises per module (40 total), five module reviews per module (25 total), and five project specifications. Create five additional edge/failure cases of your own for each module beyond the provided examples. Those25 learner-designed cases are not a prewritten test bank. Save a prediction, actual result and explanation for each.

Use 0–2 reasoning-based scoring for short reviews, and judge the substantial exercises by correct contracts, reasoning and evidence rather than vocabulary alone. The answer key offers worked solutions and review criteria; different sound designs may satisfy the same requirement.

Module 11 — Native programming with C++

Lab 11.1 — Compilation, linking and build systems

Original assignment: Build the included C++ starter, then split one function into a header and source file. Pass: reproduce and explain one compile error and one link error, then restore a warning-clean build.

Student seed: `code/starter/lesson_11_1.cpp`

Reference comparison: `code/reference/lesson_11_1.cpp`

Implementation stages

1. Configure and build the reference target. Save the full toolchain fingerprint and identify each build stage.
2. Recreate the two-file target in the starter directory. Change the function name consistently, then introduce and repair one declaration/definition mismatch.
3. Use the diagnostic fixtures to distinguish a compile error from a link error. Keep both messages in the journal.
4. Add a boundary test and rebuild from a new build directory. Explain why the test remains active in Release.

Required case matrix

1. ordinary input 0 -> 1
2. ordinary input 41 -> 42
3. maximum int -> overflow error before addition
4. missing declaration -> compile rejection
5. missing definition -> link rejection

Submission: Two source files and a header, reproducible commands, actual diagnostic transcripts, a warning-clean restored build, and a labeled compile/link/load diagram.

Independent transfer: A teammate supplies a declaration taking `long` but a definition taking `int`. Explain why the implementation may not satisfy the call, then repair the mismatch without a cast at the call site.

Reading: R12: Getting Started and libraries; R11: Source Files.

Lab 11.2 — Integer, floating-point and object representations

Original assignment: Print relevant sizes and numeric limits on the available compiler. Write checked conversions for a file length and tests for boundary values. Pass: do not trigger signed overflow as a test oracle; reject out-of-range input before converting.

Student seed: `code/starter/lesson_11_2.cpp`

Reference comparison: `code/reference/lesson_11_2.cpp`

Implementation stages

1. Print numeric limits, sizes, byte width and selected alignment values. Mark which are observations rather than portable constants.
2. Implement checked decimal length parsing and pre-multiplication budget checks.
3. Write a field-based little-endian encoder/decoder and compare with independently written bytes.
4. Test numeric boundaries and document the difference between Python integers and the chosen native representation.

Required case matrix

1. 0 and exact application maximum accepted
2. negative text and trailing junk rejected
3. one above the application limit rejected
4. huge decimal value rejected by parser
5. 42 records of 24 bytes accepted under 1024; 43 rejected
6. 0x12345678 encodes to 78 56 34 12

Submission: Representation report, boundary-test table, checked conversion implementation, specified encoding and a comparison with the earlier Python model.

Independent transfer: A texture claims width = 100,000 and height = 100,000. Design checks for width * height * bytes_per_pixel without overflowing during validation.

Reading: R11: arithmetic and conversion guidance; R15: information encoding and arithmetic.

Lab 11.3 — Values, references, pointers and lifetimes

Original assignment: Implement functions taking values, const references and pointers with a documented null policy. Pass: explain each lifetime and identify a deliberately supplied dangling-reference example without executing undefined behavior.

Student seed: `code/starter/lesson_11_3.cpp`

Reference comparison: `code/reference/lesson_11_3.cpp`

Implementation stages

1. Implement equivalent small functions taking a value, a const reference and a pointer with an explicit null policy.
2. Draw owners and borrowers for each function call. State what happens after a vector growth or owner destruction.
3. Implement a copied-value lookup and contrast it with a short-lived borrowed lookup.
4. Diagnose the supplied dangling example without executing it; submit the corrected version and reasoning.

Required case matrix

1. empty range -> absent
2. present ID -> copied expected position
3. missing ID -> absent
4. null optional target -> explicit result
5. copied result remains valid after source growth
6. dangling local identified by inspection

Submission: Parameter-choice table, lifetime diagrams, null-policy tests and a repaired invalid-reference example.

Independent transfer: An event callback captures a local variable by reference and runs next frame. Show two safe designs and explain which one owns the captured state.

Reading: R11: Interfaces, Resource Management and lifetime safety guidance.

Lab 11.4 — Standard containers, iterators and error handling

Original assignment: Port the unit list to std::vector and implement lookup with an explicit not-found result. Pass: test empty input, missing ID and growth; document which operations invalidate stored references.

Student seed: `code/starter/lesson_11_4.cpp`

Reference comparison: `code/reference/lesson_11_4.cpp`

Implementation stages

1. Port the unit list to a vector-backed API with unique IDs and copied optional lookup results.
2. Test empty, present and missing queries, duplicate insertion, growth and erasure.
3. Write an invalidation table for reserve, append, erase and clear. Include the past-the-end iterator.
4. Classify normal absence, bad external input and invariant failure; explain the chosen error channel for each.

Required case matrix

1. empty lookup -> absent
2. duplicate ID -> rejected with unchanged size
3. growth preserves logical ID lookup
4. missing removal -> false and unchanged state
5. erase adjacent zero-HP units correctly
6. reserve changes capacity, not constructed count

Submission: Unit repository, automated tests, error taxonomy and operation-specific invalidation table.

Independent transfer: A renderer stores vector iterators across an update that removes dead units. Redesign the frame boundary so rendering never uses invalid iterators.

Reading: R11: standard-library use and error handling; R12: testing integration.

Module 11 — eight additional exercises

E11.1 — Trace the build graph

A game executable consists of main.cpp, world.cpp and world.hpp. main includes world.hpp; world.cpp supplies update_world. Draw the preprocessing, compilation and linking stages. Identify what must rebuild after a public signature changes.

E11.2 — Define a checked budget

A texture header declares width and height as decimal unsigned values, each at most 16,384. Pixels need four bytes and the allocation budget is 64 MiB. Specify a checked validation order and give one legal and one rejected case.

E11.3 — Repair a borrowed view

A function builds a local std::string and returns std::string_view into it. Identify the owner, the lifetime end and two valid redesigns. Do not execute the defect to prove it.

E11.4 — Choose stable identity

You keep a pointer to an element of a vector of units. More units spawn, then an earlier element is erased. Design a lookup contract that does not depend on old addresses.

E11.5 — Binary encoding

Encode entity ID 0x01020304 as four little-endian bytes. Explain why writing sizeof(Unit) bytes from an object is not this format.

E11.6 — Floating-point evidence

On your compiler, print max_digits10, digits and is_iec559 for float and double. Compare (a+b)+c with a+(b+c) for a large cancellation case. What is the conclusion allowed by one run?

E11.7 — Error contracts

A resource lookup may legitimately miss, a packet may be malformed, and a developer may violate an internal invariant. Design distinct observable outcomes and tests for each.

E11.8 — Cross-language replay

Define a six-command, integer-only grid replay and run equivalent Python and C++ transitions. Include an edge hit and a no-op. Compare complete states, not just a printed final coordinate.

Module 11 project — Native deterministic grid simulation

Original project: Port the deterministic grid simulation to C++20, with a command-line build, automated tests, warnings enabled and an explicit account of representation differences from Python.

Port the learner's earlier integer grid model into a small C++20 command-line program. The bundled grid fixture is a teaching baseline, not a claim that your original game used exactly these rules.

1. Write a transition contract: initial state, dimensions, legal inputs, blocked/invalid behavior, tick ordering and output schema. Preserve the earlier model or document deliberate changes.
2. Separate declarations, implementation and tests. Use stable IDs and checked integer operations. Keep rendering outside the pure transition rule.
3. Run the same command traces through the original implementation and the port; compare every state and acceptance result. Include an invalid input without invoking undefined behavior.
4. Build from a fresh directory in Debug and Release. Save warnings, compiler/configuration and a representation-difference note.

Deliver: A reproducible native target, tests and replay fixtures, a before/after behavior table, and an explanation of at least three Python/C++ differences: integer range, object lifetime/ownership and error handling.

Accept when: Valid traces match the written model; invalid inputs follow the stated policy; numeric bounds are checked before unsafe arithmetic; stale references are not retained across mutations. A terminal replay suffices; graphics are not required.

Module 11 review

1. Explain compile, link and runtime failure using one concrete defect for each.
2. Why is cast-then-check too late for a large file length?
3. When is a returned span safe?
4. What is the difference between an entity ID and a vector index?
5. What evidence establishes the native port rather than just successful compilation?

Five new failure cases

Create five cases not copied from this module. For each, record the changed assumption/input, predicted outcome, actual result, invariant exercised and any repair. Use templates/FAILURE_CASES.md. A reviewer should be able to explain how these cases differ from the supplied suite.

Module 12 — Ownership, abstractions and C++ design

Lab 12.1 — RAII and resource ownership

Original assignment: Wrap a mock file/device resource whose open/close counters can be inspected. Inject failure after acquisition. Pass: each acquired resource is released exactly once and no view outlives its owner.

Student seed: `code/starter/lesson_12_1.cpp`

Reference comparison: `code/reference/lesson_12_1.cpp`

Implementation stages

1. Implement or inspect a mock resource with observable acquisition and release counters.
2. Wrap it in a non-copying RAII owner and test normal completion, early return and injected failure.
3. Add an enclosing object with dependent members and explain partial-construction cleanup.
4. Draw the device-before-resource construction and resource-before-device destruction order.

Required case matrix

1. successful scope releases exactly once
2. injected exception releases exactly once
3. empty owner releases nothing
4. nested owners unwind in reverse order
5. all paths return live count to baseline

Submission: Acquisition/release ledger, fault-injection tests, ownership diagram and destruction-order explanation.

Independent transfer: A file close operation can fail when flushing buffered data. Separate explicit error-reporting close from destructor cleanup without allowing two releases.

Reading: R11: Resource Management and smart-pointer ownership rules.

Lab 12.2 — Copy, move and exception safety

Original assignment: Implement a move-only mock resource and transactional asset replacement. Pass: copying is prohibited, moving does not double-release, and injected load failure preserves the old asset.

Student seed: `code/starter/lesson_12_2.cpp`

Reference comparison: `code/reference/lesson_12_2.cpp`

Implementation stages

1. Implement a move-only mock owner or explain a rule-of-zero alternative.
2. Add compile-time tests proving copying is prohibited and move construction is supported.
3. Build a transactional replacement function and inject failures before commit.
4. Measure the maximum simultaneous live-resource count and state the memory tradeoff.

Required case matrix

1. copy constructible is false
2. move leaves exactly one owning handle
3. move assignment releases prior destination
4. self-move remains valid
5. load/validation failure preserves old asset
6. successful replacement releases old asset once

Submission: Ownership transfer trace, compile-time constraints, failure matrix and a statement of basic/strong/no-throw guarantees.

Independent transfer: A reload operation updates both an asset and an ID index. Show why swapping only the asset may not provide the strong guarantee for the whole repository.

Reading: R11: class special members, rule of zero and exception safety.

Lab 12.3 — Interfaces, composition and polymorphism

Original assignment: Create interchangeable console and recording renderers behind one small interface. Pass: both satisfy the same contract, tests run without display output, and ownership/destructor behavior is explicit.

Student seed: `code/starter/lesson_12_3.cpp`

Reference comparison: `code/reference/lesson_12_3.cpp`

Implementation stages

1. Write a one-page renderer contract before either implementation.
2. Implement console and recording backends with the same unit ordering and synchronous-borrow policy.
3. Test both through the interface with empty and multiple-unit inputs.
4. Write an architecture decision comparing runtime polymorphism with composition and a static alternative.

Required case matrix

1. empty input emits no commands
2. two units preserve input order

3. recorded commands own copied data
4. derived cleanup observed through base ownership
5. simulation does not mutate renderer ownership

Submission: Interface contract, two backend implementations, shared contract tests and design decision record.

Independent transfer: A backend queues rendering asynchronously. Amend the contract to transfer owned command data or guarantee a valid lifetime until completion.

Reading: R11: Interfaces, Class Hierarchies and composition guidance.

Lab 12.4 — Templates, concepts and data-oriented choices

Original assignment: Write a constrained generic clamp or interpolation function and benchmark two simple entity layouts. Pass: test contracts, record compiler/build mode and explain why the result is workload-specific.

Student seed: `code/starter/lesson_12_4.cpp`

Reference comparison: `code/reference/lesson_12_4.cpp`

Implementation stages

1. Implement a constrained integer clamp and test its accepted types and bound policy.
2. Implement equivalent array-of-structures and structure-of-arrays updates.
3. Collect repeated Release measurements with a verified checksum and saved CSV.
4. Explain the result using only what the workload and measurement support; list one workload that could reverse it.

Required case matrix

1. below, within and above bounds
2. equal lower and upper bounds
3. reversed bounds rejected
4. same checksum for both layouts
5. zero entities handled
6. multiple sizes and repeated samples recorded

Submission: Constrained function, contract tests, benchmark manifest, raw CSV and a bounded interpretation of the measurements.

Independent transfer: The hot loop changes from x-only integration to accessing every field of a single selected entity. Predict why the layout conclusion might change, then design a fair test.

Reading: R11: Templates and Generic Programming; R15: memory hierarchy motivation.

Module 12 — eight additional exercises

E12.1 — Partial construction

An object owns resource A and then tries to acquire B, which fails. Draw which fully constructed subobjects are destroyed and whether the outer object destructor runs.

E12.2 — Move assignment

An owner contains handle 7; a second owner contains handle 9. Move-assign the second into the first. State the destination, source and release counts. Include self-move in the contract.

E12.3 — Transactional replacement

A loaded level contains a texture handle and parsed metadata. A replacement may fail during file reading, validation or allocation. Describe the commit point.

E12.4 — Shared ownership cycle

Two shared_ptr-owning nodes point to one another. External roots are removed. Explain the leak mechanism and draw a corrected ownership graph.

E12.5 — Substitutability

A base renderer promises that draw captures a command for later execution. One subclass retains a reference to a caller's temporary command instead. Does virtual dispatch make the implementation correct?

E12.6 — Concept versus invariant

Your template requires that a value type supports <. Does that establish that a comparator is a strict weak ordering? Give a counterexample to purely syntactic checking.

E12.7 — Layout experiment

Compare an array of structs with a struct of arrays for an update using only x and vx. Define a fair benchmark and a reason the result might change for another workload.

E12.8 — One-page ownership map

Draw a resource manager with a unique owner, a cache, a renderer and a temporary command list. Label every edge owning or borrowing and each invalidating action.

Module 12 project — Transactional resource manager

Original project: Build a small resource manager with explicit ownership, safe move behavior, bounded interfaces, fault-injection tests and a written comparison of composition and inheritance.

Manage mock resources so ownership, failure and replacement are observable without requiring a GPU or real device.

1. Choose a unique owner type with empty/moved-from states and a live-resource counter. State the allocator/deallocator pairing and borrowed counter lifetime.
2. Build a manager using stable keys, owned resources and bounded public operations. Reject duplicates or specify replacement behavior.
3. Acquire a replacement in temporary owned state; validate it; commit only when it is ready. Inject failure at each acquisition/validation stage.
4. Exercise scope exit, partial construction, move construction, move assignment, self-move and shutdown. Compare a composed backend with an inheritance-based interface using the same behavior contract.

Deliver: Ownership diagram, API contract, fault-injection table, lifetime/move tests and a one-page design comparison. Real graphics resources are an optional later extension, not needed here.

Accept when: Every successful acquisition has exactly one release by the end of each isolated test; failed replacement preserves the previous valid resource; copies cannot accidentally duplicate ownership; no exception crosses a destructor or a C boundary.

Module 12 review

1. Why should a resource be placed into an owner immediately?
2. Does std::move itself move the resource?
3. When can replacing an object satisfy the strong guarantee?
4. Why is a virtual destructor relevant to unique_ptr?
5. Why are layout benchmark results conditional?

Five new failure cases

Create five cases not copied from this module. For each, record the changed assumption/input, predicted outcome, actual result, invariant exercised and any repair. Use templates/ FAILURE_CASES.md. A reviewer should be able to explain how these cases differ from the supplied suite.

Module 13 — Data structures and their costs

Lab 13.1 — Complexity, arrays and linked structures

Original assignment: Implement a growable integer buffer for study. Count copied elements under doubling and one-at-a-time growth. Pass: derive the growth behavior and separately measure elapsed time without equating it to the proof.

Student seed: `code/starter/lesson_13_1.cpp`

Reference comparison: `code/reference/lesson_13_1.cpp`

Implementation stages

1. Implement an integer-only buffer with size/capacity invariants and two growth policies.
2. Instrument relocated-element counts and predict the sequence for n = 8 and n = 9.
3. Test value preservation, empty access, exact growth boundaries and a chosen capacity budget.
4. Compare counts analytically, then collect separate Release timings against std::vector.

Required case matrix

1. empty buffer has size 0
2. out-of-range access rejected
3. n=8 exact-fit moves 28; doubling moves 7
4. n=9 doubling moves 15
5. values preserved through growth
6. capacity limit checked before arithmetic

Submission: Growth trace, geometric-series argument, invariant tests and timing results separated from the proof.

Independent transfer: A game must never allocate during a frame. Explain why amortized O(1) is insufficient and design a capacity-planning policy with an explicit failure result.

Reading: R13: arrays, cost models and asymptotic analysis; R06: sums.

Lab 13.2 — Stacks, queues and ring buffers

Original assignment: Implement a bounded queue with explicit full/empty results and wraparound tests. Pass: preserve FIFO order through more than three times its capacity in operations and never read an uninitialized slot.

Student seed: `code/starter/lesson_13_2.cpp`

Reference comparison: `code/reference/lesson_13_2.cpp`

Implementation stages

1. Choose and document the count-based full/empty convention.
2. Implement bounded push/pop without dynamic allocation during operations.
3. Hand-trace wraparound at capacity three and test capacity one.
4. Differentially test a long deterministic operation sequence against std::deque and preserve the seed or trace.

Required case matrix

1. pop empty -> absent
2. push full -> false with unchanged state
3. capacity-one alternating operations
4. wraparound returns FIFO order
5. more than 3*capacity operations
6. no read from a logically empty slot

Submission: State-transition table, invariant checks, failure-policy documentation and differential-test trace.

Independent transfer: An audio queue must never block but may drop old samples. Define a different overflow policy and identify which tests must change; do not silently reuse the command-queue contract.

Reading: R13: elementary data structures; R16: later concurrency comparison.

Lab 13.3 — Hash tables and collision policies

Original assignment: Implement chaining or open addressing with collision tests and resizing. Pass: deliberately force many collisions, test missing keys and verify that distinct equal-hash keys remain distinct.

Student seed: `code/starter/lesson_13_3.cpp`

Reference comparison: `code/reference/lesson_13_3.cpp`

Implementation stages

1. Implement a restricted chaining table with insert/update/find/erase and controlled resizing.
2. Check the full invariant after every mutation.
3. Force all keys to collide and test equal-hash distinct keys, updates and deletions.
4. Compare deterministic traces against std::unordered_map and report occupancy/probe observations separately from correctness.

Required case matrix

1. missing key -> absent

2. same key update does not increase size
3. distinct equal-hash keys preserved
4. erase chain head/middle/tail
5. all entries survive resize
6. negative and zero integer keys handled

Submission: Collision-policy specification, resize reasoning, invariant checks and differential mapping tests.

Independent transfer: A mod supplies many deliberately colliding asset names. Explain why correctness tests are insufficient for a production resource-exhaustion defense.

Reading: R13: hashing and dictionary data structures.

Lab 13.4 — Trees, heaps and priority queues

Original assignment: Implement a binary min-heap with push/pop and randomized comparison to a sorted reference. Pass: validate the heap invariant after every operation and test duplicate priorities.

Student seed: `code/starter/lesson_13_4.cpp`

Reference comparison: `code/reference/lesson_13_4.cpp`

Implementation stages

1. Implement a binary min-heap with explicit empty behavior and a size budget.
2. Validate parent/child order after every mutation.
3. Compare push/pop traces with a sorted independent oracle, including duplicates.
4. Add an event-priority tie-breaker and compare heap requirements with balanced-tree requirements.

Required case matrix

1. empty pop -> absent
2. one-element pop leaves empty
3. ascending and descending insertions
4. duplicate priorities retained
5. negative priorities ordered correctly
6. invariant after every randomized operation

Submission: Index diagram, sift traces, invariant checker, differential tests and a data-structure decision table.

Independent transfer: A scheduler now needs all events whose times lie in an interval, not only the next event. Reconsider the structure and explain the changed query cost.

Reading: R13: binary search trees, balanced trees and heaps.

Module 13 — eight additional exercises

E13.1 — Amortized growth

Starting empty, compare total old-element copies for capacity growth by exactly one and by doubling through eight insertions. Include the ninth insertion.

E13.2 — A circular trace

A capacity-three reject-on-full queue receives push10, push20, pop, push30, push40, push50, pop, pop, pop. Write results, head/count changes and logical contents.

E13.3 — Collision deletion

For open addressing, keys A and B hash to the same bucket and B is placed after A. Explain why replacing A with a never-used empty marker can break lookup.

E13.4 — Hash-table adversary

A hash function returns zero for every key. What changes in expected versus worst-case lookup, and what should a correctness test demonstrate?

E13.5 — Heap does not mean sorted

Give a valid min-heap array that is not globally sorted. Explain the order guarantee and derive parent/child indices in a zero-based array.

E13.6 — Tie semantics

Events have time and insertion sequence. Define a priority rule that produces reproducible FIFO behavior among equal times and test three ties.

E13.7 — Locality versus complexity

A linked list supports O(1) insertion after a known node. Why can a vector still win when inserting a small record into a small collection?

E13.8 — Stateful differential test

Design a seeded randomized test for your map or queue against a standard container. Explain how to make a failure actionable.

Module 13 project — Measured educational container library

Original project: Implement educational versions of a dynamic array, queue, hash table and heap, each with invariants and tests. Compare them to standard containers without replacing standard code in production merely for practice.

Implement an int-only dynamic array, bounded queue, integer hash table and min-heap, then compare their semantics and measured costs with standard containers.

1. Write representation invariants and supported limits for each structure before implementing mutation. Do not claim generic allocator-aware behavior for the narrow int-only buffer.
2. Add deterministic empty/singleton/boundary/growth/collision/tie tests. Implement a checked full policy for the queue and a correct deletion policy for the map.
3. Generate bounded seeded operation sequences and compare to suitable standard containers after every operation; preserve failing traces.
4. Measure representative workloads in Release after correctness passes. Separate allocation, setup and iteration costs; report samples rather than one fastest run.

Deliver: Four educational implementations, invariant checks, differential/golden tests, complexity derivations and a comparison report including a case where the favored design loses or has an important limitation.

Accept when: All structures preserve contracts under all tested operations; benchmarks compare equivalent work; expected/amortized/worst-case claims are labeled. Standard containers remain the default for production unless measured requirements justify otherwise.

Module 13 review

1. Distinguish worst-case push from amortized push for a doubling array.
2. What happens on full in this ring buffer?
3. What assumptions support expected constant hash lookup?
4. Why can a heap not answer arbitrary membership queries in logarithmic time just from its ordering?
5. What must every randomized container test record?

Five new failure cases

Create five cases not copied from this module. For each, record the changed assumption/input, predicted outcome, actual result, invariant exercised and any repair. Use templates/ FAILURE_CASES.md. A reviewer should be able to explain how these cases differ from the supplied suite.

Module 14 — Algorithms for maps, simulation and tools

Lab 14.1 — Searching, sorting and algorithm selection

Original assignment: Implement binary search with a documented interval convention and test empty, one-element, duplicate and missing cases. Compare two sorts on random and nearly sorted inputs. Pass: verify output order and element preservation.

Student seed: `code/starter/lesson_14_1.cpp`

Reference comparison: `code/reference/lesson_14_1.cpp`

Implementation stages

1. Implement lower bound with a written half-open invariant and a separate membership wrapper.
2. Test empty, singleton, duplicate and missing targets against the standard lower-bound algorithm.
3. Implement or inspect insertion and merge sorts, checking order, multiset preservation and stability as applicable.
4. Compare random and nearly sorted inputs with saved data and repeatable Release measurements.

Required case matrix

1. empty -> insertion position 0
2. target below all -> 0
3. duplicate target -> first equivalent position
4. target above all -> size
5. sorting preserves duplicate counts
6. equal-key records preserve order for stable implementation

Submission: Binary-search proof sketch, boundary tests, sorting property tests and distribution-specific comparison.

Independent transfer: The comparator sorts asset names case-insensitively, but search uses case-sensitive comparison. Explain why the search precondition is broken.

Reading: R13: binary search, sorting and comparison models.

Lab 14.2 — Graph traversal and dependency ordering

Original assignment: Implement BFS and topological sorting. Pass: recover a shortest unweighted path, report unreachable goals, reject a cyclic dependency graph and justify the O(V+E) traversal bound for adjacency lists.

Student seed: `code/starter/lesson_14_2.cpp`

Reference comparison: `code/reference/lesson_14_2.cpp`

Implementation stages

1. Specify graph direction, vertex domain and duplicate-edge policy.
2. Implement BFS with predecessor reconstruction and explicit unreachable results.
3. Implement Kahn topological sorting and reject cyclic inputs.
4. Validate path edges and every dependency relation, then justify the bound for your chosen frontier and representation.

Required case matrix

1. source equals goal
2. isolated unreachable goal
3. diamond graph with two shortest paths
4. directed cycle
5. self-loop cycle
6. disconnected DAG includes all vertices
7. invalid vertex rejected

Submission: Annotated graphs, predecessor traces, order validator, cycle tests and representation-specific complexity argument.

Independent transfer: A planner uses a priority queue to choose the alphabetically smallest ready asset. Update its complexity analysis instead of retaining an unqualified O(V+E) claim.

Reading: R13: graph representations, BFS, DFS and topological sorting.

Lab 14.3 — Dijkstra, A* and admissible heuristics

Original assignment: Implement Dijkstra and A* against the same grid model. Pass: compare path costs on many small maps, include no-path cases and demonstrate a deliberately overestimating heuristic that can sacrifice optimality.

Student seed: `code/starter/lesson_14_3.cpp`

Reference comparison: `code/reference/lesson_14_3.cpp`

Implementation stages

1. Define neighbor rules, blocked cells, edge costs, distance bounds and tie-breaking.
2. Implement Dijkstra and A* with stale-entry handling and an explicit reopening policy.
3. Compare costs and independently validate routes over multiple small map families.
4. Demonstrate the overestimating-heuristic counterexample and record expansion/queue metrics without claiming universal speedup.

Required case matrix

1. start equals goal
2. unreachable region
3. weighted detour cheaper than direct route
4. zero-cost edge allowed
5. negative edge rejected
6. overestimating heuristic gives 10 versus optimal 2
7. A* with h=0 agrees with Dijkstra

Submission: Cost-model specification, path validator, small-map oracle, counterexample trace and comparative metrics.

Independent transfer: Add diagonal movement with cost 1 while orthogonal movement also costs 1. Replace Manhattan with a valid lower bound and justify it.

Reading: R13: shortest paths; R20: game navigation context.

Lab 14.4 — Dynamic programming, complexity limits and approximation

Original assignment: Solve a small 0/1 knapsack instance by exhaustive search and dynamic programming. Pass: compare answers, derive state count, and describe how a heuristic would be evaluated without claiming a guarantee it lacks.

Student seed: `code/starter/lesson_14_4.cpp`

Reference comparison: `code/reference/lesson_14_4.cpp`

Implementation stages

1. Define item and capacity domains and derive the include/exclude recurrence.
2. Implement exhaustive search with a small explicit item-count cap.
3. Implement dynamic programming, checking capacity allocation and value arithmetic.
4. Differentially test small instances and compare one heuristic without overstating its guarantee.

Required case matrix

1. no items -> 0
2. capacity zero -> 0
3. all items too heavy -> 0
4. (2,3),(3,4),(4,5), B=5 -> 7
5. single (2,3), B=4 -> 3, not 6
6. many deterministic small cases agree with exhaustive solver

Submission: State definition, recurrence proof, loop-order explanation, oracle comparisons and quality evaluation of a heuristic.

Independent transfer: The loadout now limits both mass and electrical power. Define the new sufficient state and explain the growth in table size before implementing it.

Reading: R13: dynamic programming; R06: reductions and asymptotic reasoning.

Module 14 — eight additional exercises

E14.1 — Half-open search proof

For lower_bound on a sorted array, state the loop invariant and show why lo+(hi-lo)/2 is preferable to (lo+hi)/2.

E14.2 — Stability as a requirement

Records (key,arrival) are (2,A),(1,B),(2,C),(1,D). Give a stable sorted result and one acceptable-looking but unstable result.

E14.3 — Dependency direction

Texture T must exist before Material M, and M before Level L. Define edge direction, a valid topological order and a cycle rejection test.

E14.4 — BFS discovery rule

A diamond graph has S->A,S->B,A->G,B->G. Compare marking visited at enqueue versus after removal.

E14.5 — An inadmissible shortcut

S->G costs10, S->A costs1, A->G costs1. Set h(A)=100,h(G)=0. Trace goal-on-pop A*.

E14.6 — Movement changes the heuristic

Explain when Manhattan distance is appropriate for a grid and why introducing cheap diagonal moves can invalidate it.

E14.7 — 0/1 versus unbounded DP

One item weighs2 and is worth3; budget is4. Show how ascending in-place capacity iteration incorrectly permits reuse in a 0/1 solver.

E14.8 — Pseudo-polynomial cost

Your knapsack DP costs O(nB). Why is this not necessarily polynomial in the input bit length, and what should a real-time tool do with huge B?

Module 14 project — Pathfinding visualizer and asset dependency planner

Original project: Build a pathfinding visualizer and asset-dependency planner. Provide correctness arguments, unreachable cases, complexity analysis and measured comparisons on multiple map families.

Produce a small inspectable map search tool and a dependency planner. A terminal grid with the selected path and visited cells is an acceptable visualizer.

1. Define directed graph indexing, edge direction, valid costs, neighbor order and graph budgets. Add unweighted BFS and prerequisite-first topological ordering with cycle rejection.
2. Add Dijkstra and A*, route reconstruction and deterministic tie rules. Derive a heuristic from movement rules and document whether reopening is supported.
3. Use weighted, blocked, disconnected, zero-cost and deliberately misleading-heuristic cases. Independently validate path edges and total cost; compare small graphs with exhaustive or Floyd-style results.
4. Display the grid/path/search counts and produce a dependency order or cycle diagnostic. Compare multiple maps and include setup cost in the report as a separate component.

Deliver: A terminal or graphical visualizer, build-order tool, correctness argument/invariants, fixtures, independent oracle tests and complexity/performance report.

Accept when: Minimum-cost claims hold only under documented assumptions; unreachable and cyclic cases are explicit; h=0 agrees with Dijkstra; a bad heuristic demonstration is labeled a counterexample, not supported optimal behavior.

Module 14 review

1. When does BFS provide minimum-cost paths?
2. What does a topological cycle failure mean for an asset build?
3. Why skip stale priority-queue records in Dijkstra?
4. What does h=0 do to A*?
5. What is required before claiming an approximation guarantee?

Five new failure cases

Create five cases not copied from this module. For each, record the changed assumption/input, predicted outcome, actual result, invariant exercised and any repair. Use templates/ FAILURE_CASES.md. A reviewer should be able to explain how these cases differ from the supplied suite.

Module 15 — Programming languages, compilers and computability

Lab 15.1 — Language paradigms and type systems

Original assignment: Write one rule in imperative and pure-functional styles, then define a small typed C-compatible interface for a mock subsystem. Pass: state allocation ownership and reject type-invalid expressions in a toy checker.

Student seed: `code/starter/lesson_15_1.cpp`

Reference comparison: `code/reference/lesson_15_1.cpp`

Implementation stages

1. Implement the same bounded damage rule in imperative and pure-functional styles and compare a full input table.
2. Implement Number/Boolean type rules for arithmetic and logical operators.
3. Write a C-compatible header with explicit ownership, null and error policies.
4. Use the later C bridge to compile an actual C implementation and call it through a C++ wrapper.

Required case matrix

1. damage below armor -> zero effective damage
2. damage exceeds remaining HP -> clamp at zero
3. negative domain input rejected
4. Number+Number accepted
5. Number+Boolean rejected
6. null output rejected without dereference

Submission: Two equivalent rule implementations, type-rule table, invalid-expression tests and a documented C boundary.

Independent transfer: A damage function reads a global random generator. Redesign its interface so a test can reproduce the same result without relying on global state.

Reading: R14: language frontend concepts; R11: interfaces and C-style boundary guidance.

Lab 15.2 — Lexing, parsing and syntax trees

Original assignment: Implement tokens, recursive-descent parsing and evaluation for numbers, parentheses and four arithmetic operators. Pass: test precedence, associativity, malformed input, unknown names and division-by-zero policy.

Student seed: `code/starter/lesson_15_2.cpp`

Reference comparison: `code/reference/lesson_15_2.cpp`

Implementation stages

1. Write the grammar, numeric domain, evaluation order and error-location policy before coding.
2. Implement a bounded lexer retaining offsets and a recursive-descent parser producing an arena AST.
3. Implement evaluation with intermediate-range and division checks.
4. Test precedence, associativity, malformed input, unknown names and all declared resource limits.

Required case matrix

1. 2+3*4 -> 14
2. (2+3)*4 -> 20
3. 8-3-1 -> 4
4. -8/3 -> -2
5. 1/(2-2) -> division error at slash
6. empty/2 3/unknown name -> diagnostic
7. excess source, tokens, nodes or depth rejected

Submission: Language specification, token/AST traces, golden-result table, located diagnostics and bounded-input tests.

Independent transfer: Add a remainder operator. Specify precedence, negative-operand behavior and zero-divisor policy before changing the lexer or parser.

Reading: R14: Kaleidoscope lexer, parser and abstract syntax tree chapters.

Lab 15.3 — Bytecode, virtual machines and bounded execution

Original assignment: Compile the expression language into stack bytecode and compare results to the tree evaluator. Pass: reject stack underflow, invalid opcodes and over-budget execution; preserve source context for errors.

Student seed: `code/starter/lesson_15_3.cpp`

Reference comparison: `code/reference/lesson_15_3.cpp`

Implementation stages

1. Specify each opcode, stack effect, operand order and error location.
2. Compile the AST in postorder and compare golden stack traces.
3. Implement VM checks for malformed instructions and resource limits.

4. Differentially test valid expressions and error behavior, explicitly documenting shared validation code.

Required case matrix

1. 8-3*2 -> 2
2. five instructions pass with fuel 5, fail with 4
3. ADD on empty -> underflow
4. unknown opcode rejected
5. two final values rejected
6. PUSH beyond stack cap rejected
7. invalid literal and arithmetic range rejected

Submission: Opcode table, stack traces, compiler/VM differential tests, malformed-bytecode tests and limit matrix.

Independent transfer: Add a conditional jump. Identify the new obligations for instruction-pointer validation, fuel, reachable stack states and source-level termination claims.

Reading: R14: code generation and execution concepts; course security requirements.

Lab 15.4 — Automata, computability and compiler optimization

Original assignment: Add constant folding to the toy compiler and test it against the unoptimized evaluator. Pass: include a finite-state recognizer, explain a termination restriction and document transformations deliberately excluded for semantic reasons.

Student seed: `code/starter/lesson_15_4.cpp`

Reference comparison: `code/reference/lesson_15_4.cpp`

Implementation stages

1. Implement the finite-state decimal recognizer and separate lexical validity from numeric validity.
2. Add constant folding with explicit preservation of values and located runtime failures.
3. Compare original, bytecode and folded execution on golden and generated inputs.
4. Write a termination argument for the restricted language and list excluded transformations with semantic reasons.

Required case matrix

1. empty digits rejected; 007 accepted; 7x rejected
2. 2+3*4 folds to 14
3. (1/0)*0 retains division error

4. range overflow preserved as error
5. left-before-right error location retained
6. optimized and unoptimized valid results agree

Submission: State-transition table, optimizer rules, equivalence tests, preserved-failure examples and restricted-termination argument.

Independent transfer: A proposed rewrite replaces `x-x` with zero after the language adds effectful function calls. Explain which assumptions would be required before approving it.

Reading: R14: optimizer support; R06: computation models and proof limitations.

Module 15 — eight additional exercises

E15.1 — Type errors before execution

Write the rules for number addition and a Boolean condition. Classify 3+4, true+4 and if3then1else2 under your explicit toy rules.

E15.2 — Parse an associativity trap

Draw ASTs for 9-4-2 and 9-(4-2). Explain the parser control flow that produces left-associative subtraction.

E15.3 — Located diagnostics

For source 2 + (3 *), identify a useful error category and byte offset. What information should survive bytecode compilation?

E15.4 — Stack accounting

Trace PUSH8,PUSH3,SUB,PUSH2,MUL. Give stack depth after each instruction and the final value.

E15.5 — A limit is not a sandbox

Your VM has 4,096 instructions of fuel. Explain why that alone does not bound lexing, parsing, allocation or a future host callback.

E15.6 — Error-preserving optimization

Under the bounded integer language, may (1/0)*0 become0? May 2+3*4 become14? State a safe folding policy.

E15.7 — Termination reasoning

Explain why a finite straight-line bytecode program terminates under the provided model, then why the same explanation fails after adding backward jumps.

E15.8 — Independent cross-check

A tree evaluator and VM both use the same checked arithmetic helper. What bugs can differential comparison miss, and how do you strengthen evidence?

Module 15 project — Gate C integration and bounded expression runtime

Original project: Complete Gate C: a tested C++ simulation plus a tiny expression language with lexer, parser, evaluator, bytecode execution and resource limits.

Integrate the native simulation, ownership evidence, container/algorithm comparisons and a tiny lexer/parser/tree evaluator/bytecode VM. This is Gate C, not an additional independent sixth project.

1. Implement the exact grammar and bounded arithmetic semantics. Keep located tokens and AST ownership explicit; reject unknown names and malformed syntax without partial execution.
2. Compile ASTs to checked stack instructions. Compare tree and VM results and errors, then add successful-subtree constant folding and a finite-state recognizer.
3. Enforce source, token, node, depth, bytecode, stack and fuel budgets. Test every rejection path and document the restricted trust boundary.
4. Complete the four C bridge exercises, integrate a deterministic simulation demo and prepare the Gate C review dossier described in CAPSTONE_SPEC.pdf.

Deliver: A student-built Gate C repository, reproducible tests and demo, language specification, resource-bound table, C bridge evidence, design report and oral-review notes.

Accept when: Use the detailed Gate C rubric and non-negotiable checks. Running supplied reference examples alone is not project completion. Libraries may support the work but may not silently replace the central educational implementations.

Module 15 review

1. Separate syntax, typing and execution with an example.
2. Why keep source offsets in tokens and instructions?
3. Why reject final VM stack depth other than one?
4. What changes when you add host callbacks?
5. Why is a matching optimized result on one expression insufficient?

Five new failure cases

Create five cases not copied from this module. For each, record the changed assumption/input, predicted outcome, actual result, invariant exercised and any repair. Use templates/FAILURE_CASES.md. A reviewer should be able to explain how these cases differ from the supplied suite.

Sources and provenance

Continuity with the original program

This expansion was prepared for the September28,2026 curriculum edition. The original program manifest identifies Volume03 as C++, Algorithms and Language Implementation, Modules11–15, GateC and400–650 planning hours. Its lesson IDs, titles, outcomes, prerequisites, practical assignments and assigned readings are preserved in `source/ORIGINAL_VOLUME_03_MANIFEST.json`. The 60 original quiz questions and answers were recovered from the original `CURRICULUM.html`; the expansion adds40 application questions without replacing the originals.

Teaching explanations, worked examples, code, additional exercises, reviews, bridge material and project elaborations are original instructional synthesis for this bundle. Primary reading references support the underlying concepts. No external textbook, paper, compiler distribution, proprietary SDK or font file is included. Linked resources retain their own terms. Current documentation was consulted on September28,2026; historical course archives are intentionally used for durable foundations. Not every section of every historical source was re-reviewed in full.

The original assignments are preserved even where the starter supplies only a first contract. The full lab remains a learner requirement. Complete reference examples provide comparison evidence, not pre-completed module projects. The source JSON distinguishes original questions from additions.

Primary reference directory

R06 — 6.042J Mathematics for Computer Science

MIT OpenCourseWare. Proofs, graphs, counting and discrete probability.

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

R11 — C++ Core Guidelines

C++ Core Guidelines editors. Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R12 — CMake tutorial

Kitware. Build systems; use documentation matching the installed release.

https://cmake.org/cmake/help/latest/guide/tutorial/index.html

R13 — 6.006 Introduction to Algorithms

MIT OpenCourseWare. Data structures, graph algorithms and dynamic programming.

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

R14 — Kaleidoscope and LLVM tutorials

LLVM Project. Compiler frontends, intermediate representation and JIT compilation.

https://llvm.org/docs/tutorial/

R15 — 6.004 Computation Structures

MIT OpenCourseWare. Logic, processor design, memory and systems.

https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

R16 — Operating Systems: Three Easy Pieces

Remzi and Andrea Arpaci-Dusseau. Virtualization, concurrency and persistence; book by its authors.

https://pages.cs.wisc.edu/~remzi/OSTEP/

R20 — 15-466/666 Computer Game Programming

Carnegie Mellon University. Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

P4-01 — CMake compile features and language standards

Kitware. Target requirements, language standards and extensions. Use the documentation matching your CMake release.

https://cmake.org/cmake/help/latest/manual/cmake-compile-features.7.html

P4-02 — What Every Computer Scientist Should Know About Floating-Point Arithmetic

David Goldberg / Oracle documentation. Author paper reproduced in official numerical-computation documentation; rounding and non-associativity background.

https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

P4-03 — AddressSanitizer

LLVM / Clang. Compiler/runtime setup and kinds of executed memory errors detected. Detection is not an all-input proof.

https://clang.llvm.org/docs/AddressSanitizer.html

P4-04 — UndefinedBehaviorSanitizer

LLVM / Clang. Runtime instrumentation and supported checks. Platform/toolchain support must be verified.

https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

P4-05 — Language standards supported by GCC

GNU Compiler Collection. C/C++ standards and language modes. This course selects C17/C++20, not a newest-standard claim.

https://gcc.gnu.org/onlinedocs/gcc/Standards.html

P4-06 — Kaleidoscope: implementing a parser and AST

LLVM Project. Frontend architecture reading; the course language and integer semantics are independently specified and do not require LLVM libraries.

https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/LangImpl02.html

P4-07 — Fundamental types [basic.fundamental]

C++ working-draft editors / eel.is rendering. Current working-draft view, not the fixed published C++20 edition. Consult only the stable integer/representation concepts used here; later additions are outside this course baseline.

https://eel.is/c++draft/basic.fundamental

P4-08 — Vector modifiers, capacity and unordered-container requirements

C++ working-draft editors / eel.is rendering. Companion sections: https://eel.is/c++draft/vector.capacity and https://eel.is/c++draft/unord.req. Used for operation-specific invalidation rules; current draft contains later features not used by the course.

https://eel.is/c++draft/vector.modifiers

Reading by topic

Use R11 for ownership, interfaces, special members, exception safety and source-file organization; R12/P4-01 for builds; R13 for arrays, hashing, heaps, sorting, graphs and dynamic programming; R14/P4-06 for language frontends and compiler structure. R06 supports discrete proof and computability reasoning, and R15 connects representations to later hardware study. R16/R20 are cross-volume reading pointers, not dependencies of the supplied build.

Read the exact assignment printed under each lesson. Record the section actually studied and at least one exercise or derivation in the journal. The narrow course implementation deliberately differs from some reading examples: bounded integers instead of general floating-point language values, a small straight-line stack machine instead of an LLVM JIT, and mock resources instead of a graphics API. Do not merge incompatible assumptions silently.